

STAT3003: 统计算法基础

优化问题 (2)

曾靖

中国科学技术大学管理学院

大纲

大纲

1. 二分法
2. 牛顿法
3. 梯度下降法

本节内容来自于《统计计算使用 R 》第 11 章和《统计计算》第六章。

1 引言

1.1 优化问题

- 在统计计算中广泛用到求最小（最大）值点。如
 - 极大似然估计 (MLE)
 - 最小二乘估计 (OLS)
- 最大值问题 $\max_{x \in \mathbb{R}^d} f(x)$ 和最小值问题 $\min_{x \in \mathbb{R}^d} f(x)$ 统称为优化问题。
- 最大值问题总能转化为最小值问题，即 $\max_{x \in \mathbb{R}^d} f(x) = -\min_{x \in \mathbb{R}^d} \{-f(x)\}$ 。因此，我们只讨论最小值问题。

- 一般而言，优化问题没有显式解，这时候需要通过迭代算法 (*iterative algorithm*) 来进行数值求解。也就是从上一步的近似值 x_t 通过迭代公式得到下一步的近似值 x_{t+1} 。
- 由于是迭代算法，实际中需要找到合适的停止法则 (*stopping criteria*) 来停止算法的迭代。当满足停止法则时算法停止，并输出结果。令 $\varepsilon > 0$ 为一个很小的常数，通常称为 **tolerance**，常用的停止法则如下：
 - 绝对收敛： $\|x_{t+1} - x_t\| < \varepsilon$;
 - 相对收敛： $\|x_{t+1} - x_t\| / \|x_t\| < \varepsilon$;
 - 其他收敛： $\|f'(x)\| < \varepsilon$ 。
- ε 并不是越小越好，实际数值计算中，函数取值往往只有一定的精度，过小的 *tolerance* 会导致算法在真实解的附近反复跳跃而不能收敛。另一方面，为了保险起见，我们最好设置一个最大迭代次数，防止算法失败导致无法停止。

1.2 收敛速率

- 记优化问题第 t 步的变量为 x_t , 设

$$\lim_{t \rightarrow \infty} x_t = x^*,$$

我们通常用 Q (*Quotient*)-收敛和 R (*Root*)-收敛来刻画算法的收敛速率。

1.2 收敛速率

Q -收敛: 记

$$Q = \overline{\lim}_{t \rightarrow \infty} \frac{\|x_{t+1} - x^*\|}{\|x_t - x^*\|},$$

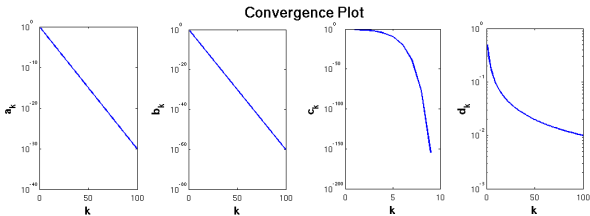
若 $Q = 0$ 则算法具有超线性收敛速率, 若 $0 < Q < 1$ 则算法具有线性收敛速率, 若 $Q = 1$ 则称算法具有次线性收敛速率。下面给出一些具体例子

- $\|x_t - x^*\| = O(\exp(-\exp(t)))$, 则 $Q = 0$;
- $\|x_t - x^*\| = O(\exp(-t))$, 则 $0 < Q < 1$;
- $\|x_t - x^*\| = O(1/t)$, 则 $Q = 1$;
- 在超线性收敛的基础上, 如果存在 $p > 1$ 使得

$$\overline{\lim}_{t \rightarrow \infty} \frac{\|x_{t+1} - x^*\|}{\|x_t - x^*\|^p} = C < \infty,$$

则称算法依 p 阶收敛到 x^* 。

- 到达一定精度 $\varepsilon > 0$ 需要的迭代次数来经常被用来刻画收敛速度：
 - 超线性收敛: $O(\log(\log(1/\varepsilon)))$;
 - 线性收敛: $O(\log(1/\varepsilon))$;
 - 次线性收敛: $O(1/\varepsilon)$.



- y 轴 (对数刻度): $\|x_k - x^*\|$
- x 轴: k
- 从左到右分别展示线性，线性，超线性，次线性。

1.2 收敛速率

R -收敛: 如果 Q 不存在, 我们利用另一个收敛序列来刻画其收敛性, 称为 R 收敛。

- 假设 $\|x_t - x^*\| \leq y_t$, 如果序列 $\{y_t\}$ 是线性收敛, 则称 $\{x_t\}$ 是 R -线性收敛。相应地有 R -超线性收敛, R -次线性收敛。
 - 例如 $y_t = q^t, 0 < q < 1$.

2 一维问题中的优化算法

2.1 二分法求根

- 优化问题与求根问题密切相关，很多优化问题会归结为一个求根问题。例如，若一元目标函数 $f(x)$ 连续可微，则极值点一定是 $g(x) = f'(x) = 0$ 的根。
- 理论根据：介值定理
 - 设函数 $g(x)$ 在闭区间 $[a, b]$ 上连续并且 $g(a)g(b) < 0$ ，则由介值定理可知至少存在一个点 $x^* \in [a, b]$ 使得 $g(x^*) = 0$ ，即 x^* 为 $g(x)$ 的根。如果 $g(x)$ 在 $[a, b]$ 上是严格单调函数，则根 x^* 存在且唯一。

算法:

- Input: 函数 $g(x)$, 左右端点 a, b 。令 $g_a = g(a), g_b = g(b)$ 。
- Loop: 重复一下步骤直到满足某停止法则为止
 1. (计算中点函数值) 令 $c = \frac{1}{2}(a + b), g_c = g(c)$ 。
 2. (更新区间) 如果 $g_a g_c \leq 0$, 则令 $b = c$; 否则, 令 $a = c$ 。
- Output: c

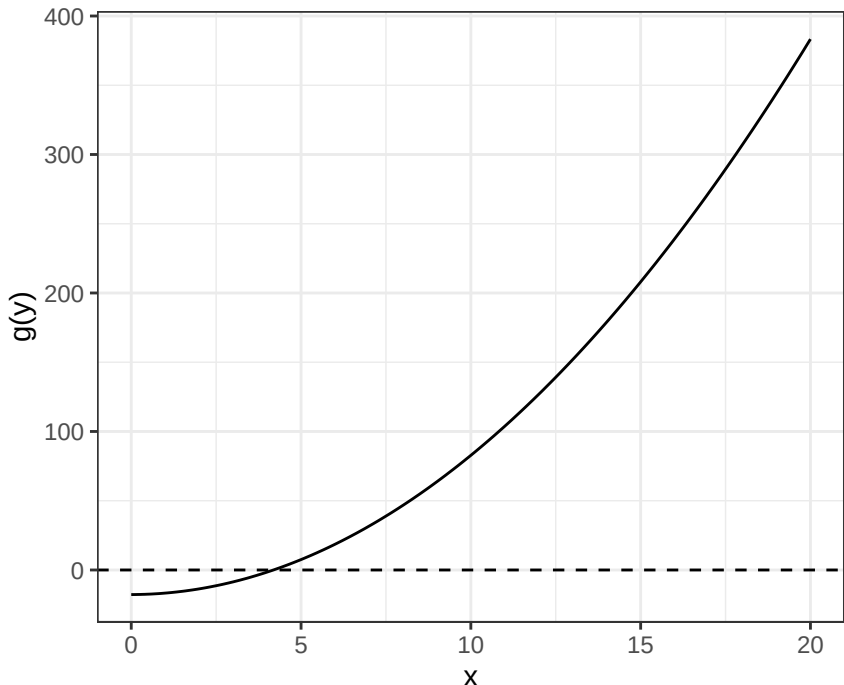
例 2.1

求解方程

$$a^2 + y^2 + \frac{2ay}{n-1} = n - 2.$$

- 令 $g(y) = a^2 + y^2 + \frac{2ay}{n-1} - (n - 2)$.

```
g0 <- function(y, a = 0.5, n=20){  
  a^2 + y^2 + 2*a*y/(n-1) - (n-2)  
}  
g0_data <- data.frame(x = seq(0, 20, length.out = 301))  
ggplot(g0_data, aes(x = x, y = g0(x))) +  
  geom_line() + geom_hline(yintercept = 0, linetype = 2) +  
  labs(x = "x", y = "g(y)") + theme_bw()
```



```
# Definition of function g
g <- function(y, a, n) {
  a^2 + y^2 + 2*a*y/(n-1) - (n-2)
}

a <- 0.5
n <- 20
b0 <- 0 # left endpoint
b1 <- 5*n # right endpoint
it <- 0 # iteration
eps <- .Machine$double.eps^0.25 # tolerance
```

#solve using bisection

```
r <- seq(b0, b1, length=3)
```

```
y <- c(g(r[1], a, n), g(r[2], a, n), g(r[3], a, n))
```

```
if (y[1] * y[3] > 0)
```

```
  stop("g does not have opposite sign at endpoints")
```

```
track <- c(r[2], y[2])
```

```
while(it < 1000 && abs(y[2]) > eps) {
```

```
  it <- it + 1
```

```
  if (y[1]*y[2] < 0) {
```

```
    r[3] <- r[2]
```

```
    y[3] <- y[2]
```

```
  }else{
```

```
    r[1] <- r[2]
```

```
    y[1] <- y[2]
```

```
  }
```

```
  r[2] <- (r[1] + r[3]) / 2
```

```
  y[2] <- g(r[2], a=a, n=n)
```

```
  track <- rbind(track, c(r[2], y[2]))
```

```
}
```

```
r[2] # root
```

```
## [1] 4.186845
```

```
y[2] # value at root
```

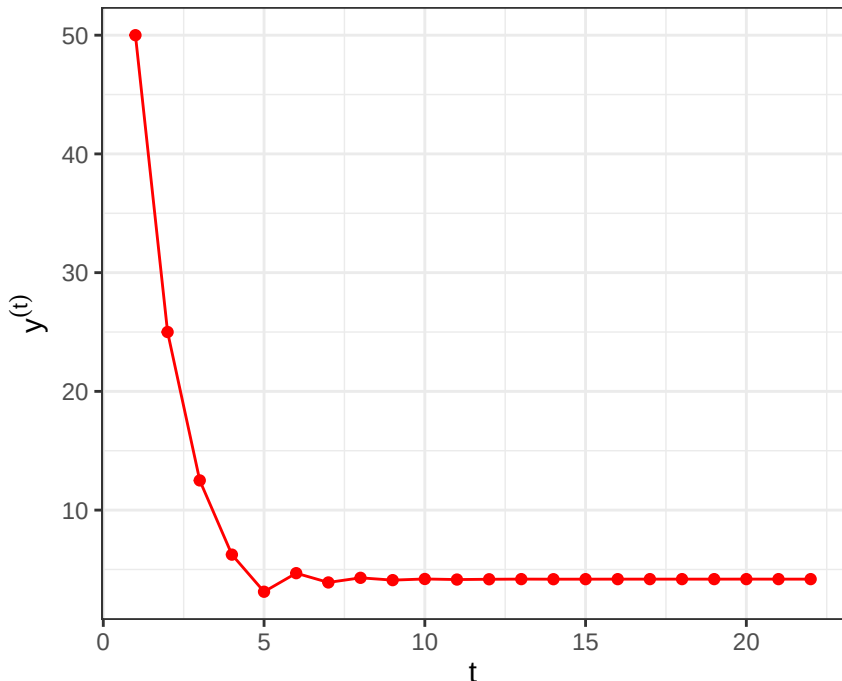
```
## [1] 2.984885e-05
```

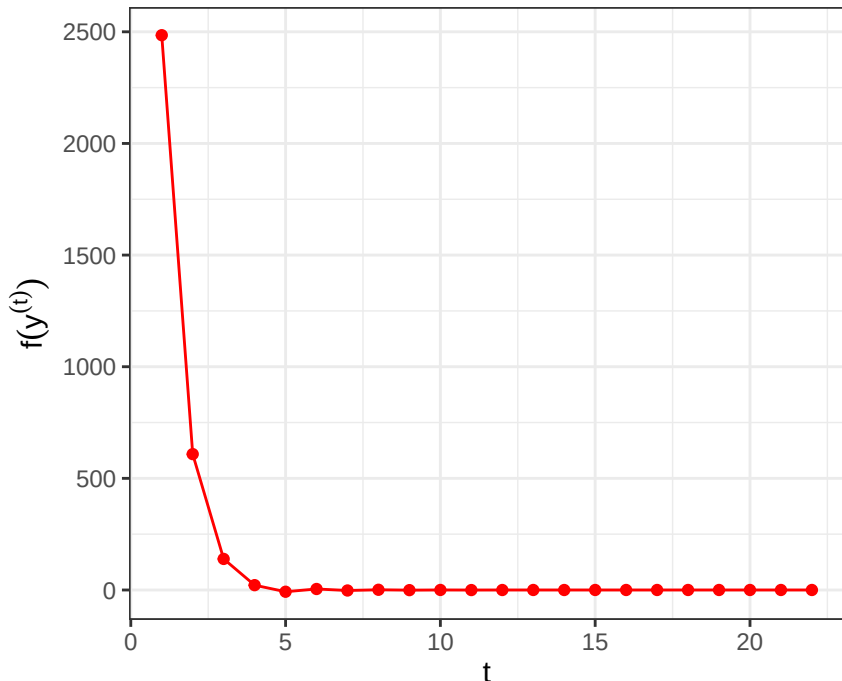
```
it # the number of iterations
```

```
## [1] 21
```

```
library(latex2exp)
ggplot_series(track[,1], xlab = TeX("$t$"),
              ylab = TeX("$y^{\{t\}}$"), color = "red")

ggplot_series(track[,2], xlab = TeX("$t$"),
              ylab = TeX("$f(y^{\{t\}})$"), color = "red")
```





2.2 一维问题中的牛顿法

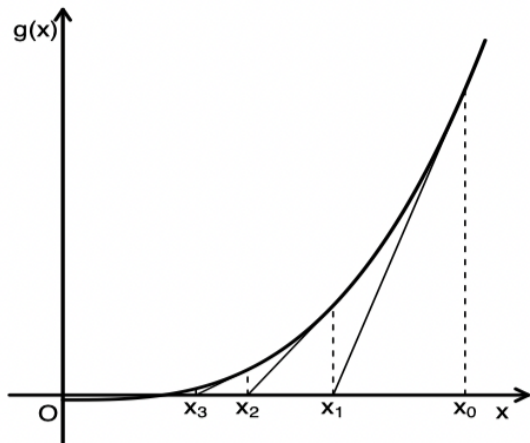
牛顿法 (*Newton's method or Newton–Raphson method*) 是一种求根方法

- 设目标函数 $g(x)$ 有连续一阶导数, 我们希望得到 $g(x)$ 的根。
- 记根为 x^* , 设 x_0 是 x^* 附近的一个点, 在 x_0 附近对 $g(x)$ 作一阶 Taylor 展开近似得

$$g(x) \approx g(x_0) + g'(x_0)(x - x_0).$$

- 令 $g(x) = 0$, 则近似解为 $x_1 = x_0 - g(x_0)/g'(x_0)$ 。
- 我们迭代地更新每一步解:

$$x_{t+1} = x_t - \frac{g(x_t)}{g'(x_t)}, \quad t = 0, 1, 2, \dots$$



- 如果 $g'(x)$ 很难推导，可以利用割线法。简而言之，用 $(x_{t-1}, g(x_{t-1}))$ 和 $(x_t, g(x_t))$ 连线的斜率代替 $g'(x_t)$ 。迭代公式为

$$x_{t+1} = x_t - \frac{x_t - x_{t-1}}{g(x_t) - g(x_{t-1})} g(x_t), \quad t = 0, 1, 2, \dots$$

- 如果问题为求解连续二阶可导函数 $f(x)$ 的最小值点，则问题等价于求解 $g(x) = f'(x) = 0$ 的根。因此，Newton 法求解 $f(x)$ 的最小值点的迭代公式为

$$x_{t+1} = x_t - \frac{f'(x_t)}{f''(x_t)}, \quad t = 0, 1, 2, \dots$$

3 多维问题中的优化算法

3.1 牛顿法

- 考虑多维空间 $\mathbb{R}^p$ 中的最小值问题:

$$x^* = \arg \min_{x \in \mathbb{R}^p} f(x)$$

- 类似于一维问题, 对于多维问题中的求解最小值问题, 假设目标函数 f 二阶连续可导, 则迭代公式为

$$x_{t+1} = x_t - (\nabla^2 f(x_t))^{-1} \nabla f(x_t),$$

这里 $\nabla^2 f(x_t) \in \mathbb{R}^{p \times p}$ 为 f 在 x_t 的 Hessian 阵。

- 在一维问题中, 牛顿法是梯度下降法的特例。

- 优点：在一些正则化条件下，如果初始值接近于最小值点，则牛顿法有二阶收敛速率。即

$$\overline{\lim}_{t \rightarrow \infty} \frac{\|x_{t+1} - x^*\|}{\|x_t - x^*\|^2} = C < \infty.$$

- 缺点：
 - 每次迭代中需要计算 Hessian 阵的逆，使得牛顿法计算量非常大；
 - 牛顿法的性能十分依赖 Hessian 矩阵 $\nabla^2 f(x_t)$ ，如果 $\nabla^2 f(x_t)$ 不满秩，则解不唯一，算法无法进行。

牛顿法的改进（一）

- 当 Hessian 矩阵不可逆，通过给 $\nabla^2 f(x_t)$ 加一个正定阵来保证正定型，即 $\nabla^2 f(x_t) \leftarrow \nabla^2 f(x_t) + \lambda I$ ，这里 $\lambda > 0$ 为某个常数。（思考：与 *ridge regression* 有相似之处）

例 3.1

- 考虑具有共线性的回归模型：

$$y = x_1 + 2x_2 + 3(x_1 + x_2) + \varepsilon.$$

- 我们用调整过 Hessian 矩阵的 Newton's method 来计算 OLS 估计量。

```
x <- matrix(rnorm(200), 100, 2)
x <- cbind(x, x[,1]+x[,2])
svd(x)$d
```

```
## [1] 1.603105e+01 9.277932e+00 1.448787e-15
```

```
y <- x%%1:3 + rnorm(100)
iter <- 0
b <- rep(0,3)

while(norm(t(x)%*(x%%b-y))>1e-3 & iter < 100){
  iter <- iter + 1
  b <- b - solve(crossprod(x) + diag(0.00001,3))%*%
    (crossprod(x)%*%b-crossprod(x, y))
  print(b)
}
```

```
##           [,1]
## [1,] 1.034507
## [2,] 1.955495
## [3,] 2.990003
```

牛顿法的改进（二）：拟牛顿法

- 在牛顿法的每一步迭代 $x_{t+1} = x_t - (\nabla^2 f(x_t))^{-1} \nabla f(x_t)$ 中，将 Hessian 阵 $\nabla^2 f(x_t)$ 换成一个正定的矩阵 $\mathbf{H}_t$.
- $\mathbf{H}_t$ 是迭代得到的。 $\mathbf{H}_{t+1} = \mathbf{H}_t + \Delta_t$, 其中 Δ_t 称为修正矩阵；
- 并且在迭代中也考虑步长，即求得

$$s_t = \operatorname{argmin}_{s>0} f(x_t - s\mathbf{H}_t^{-1} \nabla f(x_t)).$$

- 这样的方法称为拟牛顿法。

◦ 算法如下:

1. 寻找最优步长: $s_t = \operatorname{argmin}_{s>0} f(x_t - s\mathbf{H}_t^{-1}\nabla f(x_t));$
2. 更新 x_t : $x_{t+1} = x_t - s_t\mathbf{H}_t^{-1}\nabla f(x_t);$
3. 更新修正矩阵: $\Delta_t;$
4. 更新矩阵 $\mathbf{H}_t$: $\mathbf{H}_{t+1} = \mathbf{H}_t + \Delta_t.$

- 迭代地构造每一步的矩阵 $\mathbf{H}_t$, 并使得其满足如下条件:
 - $\mathbf{H}_t$ 是正定矩阵 (思考: 为什么?)
 - $\mathbf{H}_t$ 是迭代得到的。 $\mathbf{H}_{t+1} = \mathbf{H}_t + \Delta_t$, 其中 Δ_t 称为修正矩阵;
 - $\mathbf{H}_{t+1}$ 满足 Newton 条件:

$$\mathbf{H}_{t+1} \delta_t = \zeta_t,$$

这里 $\delta_t = x_{t+1} - x_t$, $\zeta_t = \nabla f(x_{t+1}) - \nabla f(x_t)$. (思考: 为什么?)

拟牛顿算法之一: *BFGS*(Broyden-Fletcher-Goldfarb-Shanno) 算法

- 修正矩阵为

$$\Delta_t = \frac{\zeta_t \zeta_t^\top}{\zeta_t^\top \delta_t} - \frac{(\mathbf{H}_t \delta_t)(\mathbf{H}_t \delta_t)^\top}{\delta_t^\top \mathbf{H}_t \delta_t},$$

这里 $\delta_t = x_{t+1} - x_t$, $\zeta_t = \nabla f(x_{t+1}) - \nabla f(x_t)$.

- 看上去在拟牛顿法算法中我们仍然需要计算 $\mathbf{H}_t^{-1}$ 。事实上, 在 *BFGS* 算法中, $\mathbf{H}_{t+1}^{-1}$ 仍然可以迭代地从 $\mathbf{H}_t^{-1}, \zeta_t, \delta_t$ 得到。记 $\mathbf{V}_t = \mathbf{H}_t^{-1}$, 则

$$\mathbf{V}_{t+1} = \mathbf{V}_t + \frac{(\zeta_t^\top \delta_t + \zeta_t^\top \mathbf{V}_t \zeta_t)(\delta_t \delta_t^\top)}{(\zeta_t^\top \delta_t)^2} - \frac{(\mathbf{V}_t \zeta_t \delta_t^\top) + (\mathbf{V}_t \zeta_t \delta_t^\top)^\top}{\zeta_t^\top \delta_t}$$

◦ (更新后的) BFGS 算法为:

1. 寻找最优步长: $s_t = \operatorname{argmin}_{s>0} f(x_t - s\mathbf{V}_t \nabla f(x_t));$
2. 更新 x_{t+1} : $x_{t+1} = x_t - s_t \mathbf{V}_t \nabla f(x_t);$
3. 计算 ζ_t 和 δ_t ;
4. 更新矩阵 $\mathbf{V}_{t+1}$

拟牛顿法特例：阻尼牛顿法

- 如果在拟牛顿法中保持 $\mathbf{H}_t = \nabla^2 f(x_t)$ ，则称该方法为阻尼牛顿法。
- 算法如下：
 1. 寻找最优步长: $s_t = \operatorname{argmin}_{s>0} f(x_t - s(\nabla^2 f(x_t))^{-1} \nabla f(x_t))$;
 2. 更新: $x_{t+1} = x_t - s_t(\nabla^2 f(x_t))^{-1} \nabla f(x_t)$.

3.2 Fisher 得分法

- Fisher 得分法是 Newton 法在 MLE 问题中的应用。
- 回顾：在牛顿法中，迭代公式为

$$\theta_{t+1} = \theta^{(t)} - (\nabla^2 f(\theta^{(t)}))^{-1} \nabla f(\theta^{(t)}).$$

- MLE 问题中的牛顿法：

$$\theta_{t+1} = \theta^{(t)} - (\nabla^2 \ell(\theta^{(t)}))^{-1} \nabla \ell(\theta^{(t)}).$$

- 因此，每次迭代中都需要计算一个 $p \times p$ 矩阵的逆

$$-(\nabla^2 \ell(\theta^{(t)}))^{-1}.$$

这往往是牛顿法最耗费算力的地方。

Fisher 信息矩阵

$$\begin{aligned} I_n(\theta) &= \text{Cov} \left(\frac{\partial \ell(\theta)}{\partial \theta} \right) \\ &= E \left\{ \left(\frac{\partial \ell(\theta)}{\partial \theta} \right) \left(\frac{\partial \ell(\theta)}{\partial \theta} \right)^\top \right\} \\ &\quad - \left\{ E \left(\frac{\partial \ell(\theta)}{\partial \theta} \right) \right\} \left\{ E \left(\frac{\partial \ell(\theta)}{\partial \theta} \right) \right\}^\top. \end{aligned}$$

- Fisher 信息阵可以用来度量随机变量 X 所携带的关于参数 θ 的信息。Fisher 信息阵越小，意味着似然函数关于 θ 越平整，包含 θ 的信息也就越少。

- 在一些正则条件下，可得

$$E \left(\frac{\partial \ell(\theta)}{\partial \theta} \right) = 0.$$

因此，

$$I_n(\theta) = E \left\{ \left(\frac{\partial \ell(\theta)}{\partial \theta} \right) \left(\frac{\partial \ell(\theta)}{\partial \theta} \right)^\top \right\}.$$

- 另外，在一些正则条件下，

$$I_n(\theta) = -E \left\{ \frac{\partial^2 \ell(\theta)}{\partial \theta \partial \theta^\top} \right\}.$$

- 回顾：在牛顿法的每次迭代中，我们需要

$$-\nabla^2 \ell(\theta^{(t)}) = - \left(\frac{\partial^2 \ell(\theta)}{\partial \theta \partial \theta^\top} \Big|_{\theta=\theta^{(t)}} \right).$$

- 当样本量足够大且 $\theta^{(t)}$ 接近真值时，我们有近似结果

$$- \left(\frac{\partial^2 \ell(\theta)}{\partial \theta \partial \theta^\top} \Big|_{\theta=\theta^{(t)}} \right) \approx I_n(\theta^{(t)}).$$

注意，等式左边是在给定 n 个样本下的值，等式右边则是一个期望。

- 因此，在 MLE 问题中，我们可以利用 Fisher 信息阵来近似替代 Newton 法中的 Hessian 阵。

- Fisher 得分法解 MLE 问题的迭代公式:

$$\theta_{t+1} = \theta^{(t)} + (I_n(\theta^{(t)}))^{-1} \nabla \ell(\theta^{(t)}).$$

- 在 Fisher 得分法中, 计算往往较为简单: 我们只需要计算出得分向量 $\nabla \ell(\theta)$ 和 Fisher 信息阵 $I_n(\theta)$ 。
- 另一方法, Fisher 得分法广泛运用于广义线性模型 (GLM) 下的 MLE 问题, 迭代公式较为简洁, 可以诱导出 IRLS (Iteratively Reweighted Least Squares) 算法。

例 3.2: MLE in logistic regression

- 假设 Y_i 满足 Bernoulli 分布 $Ber(p_i)$, $i = 1, 2, \dots, n$, 其中 $p_i = \Pr(Y_i = 1)$ 。自变量 $X_i \in \mathbb{R}^p$ 。
- 假设 p_i 和线性组合 $\beta^\top X_i$, $\beta \in \mathbb{R}^p$, 满足如下公式

$$p_i = \frac{\exp(\beta^\top X_i)}{1 + \exp(\beta^\top X_i)}.$$

- 考虑 i.i.d. 样本 (X_i, Y_i) , $i = 1, \dots, n$, 对数似然函数为

$$\begin{aligned}\ell(\beta) &= \sum_{i=1}^n \log\{p_i^{y_i} (1 - p_i)^{1-y_i}\} \\ &\propto \sum_{i=1}^n y_i \cdot \beta^\top x_i - \log(1 + \exp(\beta^\top x_i))\end{aligned}$$

- Q : 求关于参数 β 的 MLE。

- 梯度和 Hessian 矩阵分别为

$$\nabla \ell(\beta) = \sum_{i=1}^n (y_i - p_i) x_i, \quad \nabla^2 \ell(\beta) = - \sum_{i=1}^n p_i (1 - p_i) x_i x_i^\top,$$

- 由于 Hessian 矩阵不含随机成分 (x_i 样本固定), 因此牛顿法和 Fisher 得分法是相同的。

Data generation

n <- 1000

x <- rnorm(n)

beta <- 0.3

prob <- exp(x*beta)/(1+exp(x*beta))

y <- rbinom(n, 1, prob = prob)

Initialization

iter <- 0

b <- 0.3

g <- 1

```
while(abs(g) > sqrt(.Machine$double.eps) && iter < 100){  
  iter <- iter + 1  
  p <- exp(x*b)/(1+exp(x*b))  
  h <- -sum(p*(1-p)*x^2)  
  g <- sum((y-p)*x)  
  b <- b - h^-1*g  
  
  cat(iter, b, abs(g), "\n")  
}
```

```
## 1 0.2900838 2.576295  
## 2 0.2901061 0.005821046  
## 3 0.2901061 2.898313e-08  
## 4 0.2901061 1.69309e-14
```

3.3 梯度下降法 (Gradient descent)

- 一般地, 对于一个可导函数 $f(x) : \mathbb{R}^p \rightarrow \mathbb{R}$, 给定一点 $x_t \in \mathbb{R}^p$, 我们希望找到一个方向 $u_t \in \mathbb{R}^p$, 使得沿着该方向, 函数 $f(x)$ 的取值减小。于是我们沿着方向 u_t 更新 x_t , 即

$$x_{t+1} = x_t + s_t u_t,$$

这里 s_t 表示更新的步长 (stepsize)。这种方法统称为下降方法 (descent methods)。

- 沿着方向 $u \in \mathbb{R}^p$, $f(x)$ 在点 x_t 的方向导数为

$$D_u f|_{x_t} = \langle \nabla f(x_t), u \rangle,$$

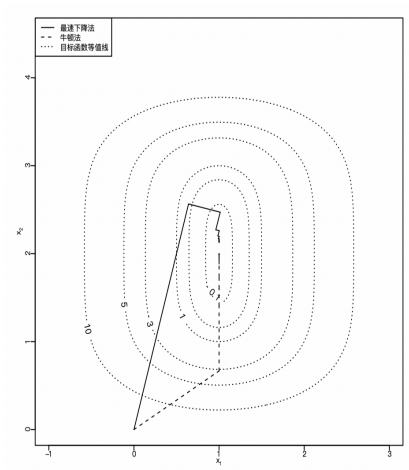
这里 ∇f 表示函数 f 的梯度。我们希望找到的方向 u 使得 $\langle \nabla f, u \rangle < 0$ 。

- 易知沿着 $-\nabla f$ 方向可以使得 f 下降最快。则迭代公式为

$$x_{t+1} = x_t - s_t \nabla f(x_t).$$

这种下降方法叫做梯度下降方法。

◦ 最速下降法的一个图示¹



¹https://en.wikipedia.org/wiki/Gradient_descent

如何选取步长?

如何选取步长会影响算法的收敛性以及收敛速率。这里我们讨论几种常见的步长选取:

- 固定步长: $s_t = s$. 在某些问题中, 固定步长会保证算法的收敛性。
- 最优步长: *limited minimization rule, Armijo or backtracking rule*。

Limited minimization rule: 在给定点 x_t 上, 沿着梯度下降方向找到是的函数值最小的步长 $s_t > 0$, 即求解

$$s_t = \operatorname{argmin}_{s>0} f(x_t - s\nabla f(x_t)).$$

Armijo (backtracking) rule: 给定参数 $\alpha \in (0, 1)$ 和 $\gamma \in (0, 1)$ 以及初始步长 $s = 1$ 。不断缩短步长 $s \leftarrow \gamma s$ 直至

$$f(x_t - s\nabla f(x_t)) \leq f(x_t) - \alpha s \langle \nabla f(x_t), \nabla f(x_t) \rangle,$$

或等价写为

$$f(x_t) - f(x_t - s\nabla f(x_t)) \geq \alpha s \langle \nabla f(x_t), \nabla f(x_t) \rangle.$$

这里, $s \langle \nabla f(x_t), \nabla f(x_t) \rangle$ 为用一阶近似估计 $f(x)$ 得到的函数减小量。因此, 我们希望在一步更新中, 函数减小量至少要达到一阶近似减小量的 α 倍。实际应用中, $\alpha = 0.5$ 和 $\gamma = 0.8$ 为合理的取值。

固定步长梯度下降算法:

- Input: 函数 f , 初始值 x_0 , 步长 s ;
- Loop: 对于 $t = 0, 1, \dots$, 重复如下步骤直到 $|f(x_t) - f(x_{t-1})| < \varepsilon$ 或者 $t \geq t_{\max}$:
 1. 更新 $x_{t+1} = x_t - s\nabla f(x_t)$.
- Output: x_t .

最优步长梯度下降算法 (*limited minimization rule*):

- Input: 函数 f , 初始值 x_0
- Loop: 对于 $t = 0, 1, \dots$, 重复如下步骤直到 $|f(x_t) - f(x_{t-1})| < \varepsilon$ 或者 $t \geq t_{\max}$:
 1. 求最佳步长 s_t 使得 $s_t = \operatorname{argmin}_{s>0} f(x_t - s \nabla f(x_t))$;
 2. 更新 $x_{t+1} = x_t - s_t \nabla f(x_t)$.
- Output: x_t .

例 3.3

最小二乘估计: 一般地, 对于 $y \in \mathbb{R}^n$, $X \in \mathbb{R}^{n \times p}$, 利用最小二乘法估计回归模型中的系数

$$\hat{\beta} = \min_{\beta \in \mathbb{R}^p} \|y - X\beta\|^2.$$

考虑一元回归模型: $y = 2x + \varepsilon$.

- 目标函数: $f(\beta) = \|y - X\beta\|^2$
- 梯度: $\nabla f(\beta) = 2X^\top(X\beta - y)$
- 迭代公式:

$$\beta_{t+1} = \beta_t - 2s_t X^\top(X\beta_t - y).$$

固定步长梯度下降法

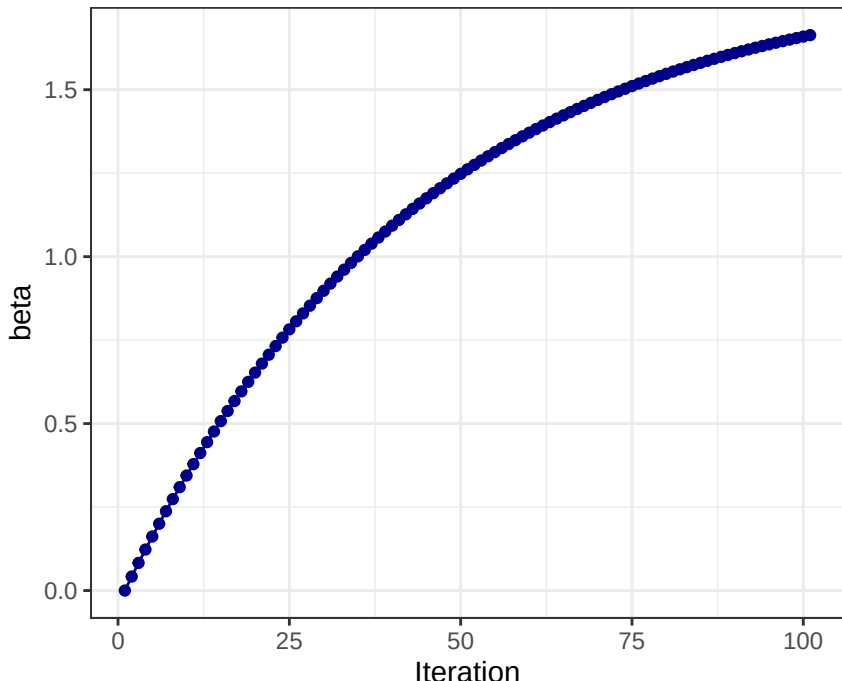
- 过小的步长

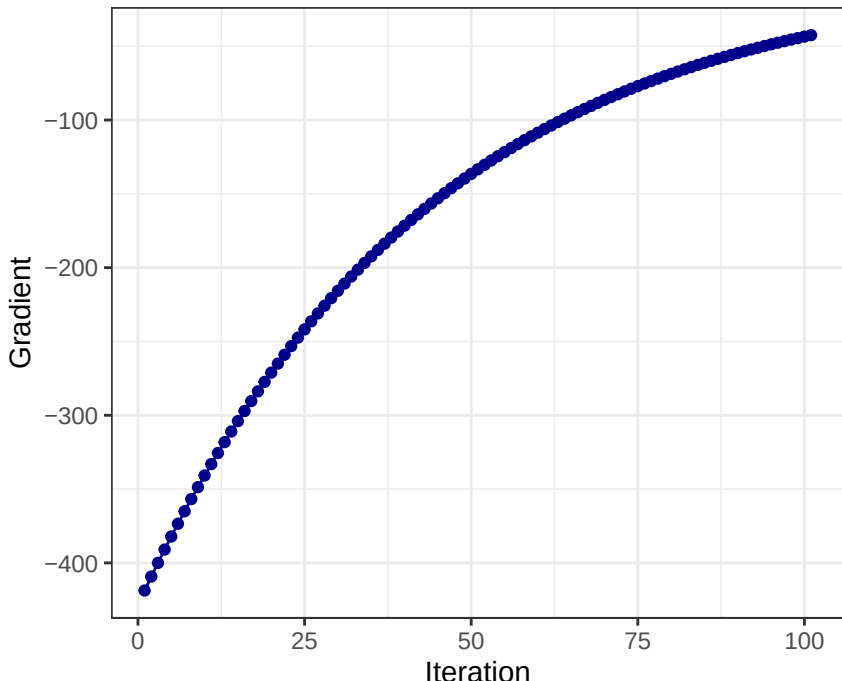
```
n <- 100
x <- rnorm(n)
eps <- rnorm(n)
y <- 2*x + eps
iter <- 0
beta <- c(0)
f <- function(beta){  # objective function
  sum((y - x * beta)^2)
}
g <- function(beta){  # gradient function
  2*(sum(x^2)*beta - sum(x*y))
}
```

```
s <- 10000{-1} # stepsize
while(abs(g(beta[iter + 1]))>1e-3 & iter < 100){
  iter <- iter + 1
  beta[iter+1] <- beta[iter] - s*g(beta[iter])
}
```

```
ggplot_series(beta, xlab = "Iteration", ylab = "beta")
```

```
ggplot_series(g(beta), xlab = "Iteration", ylab = "Gradient")
```





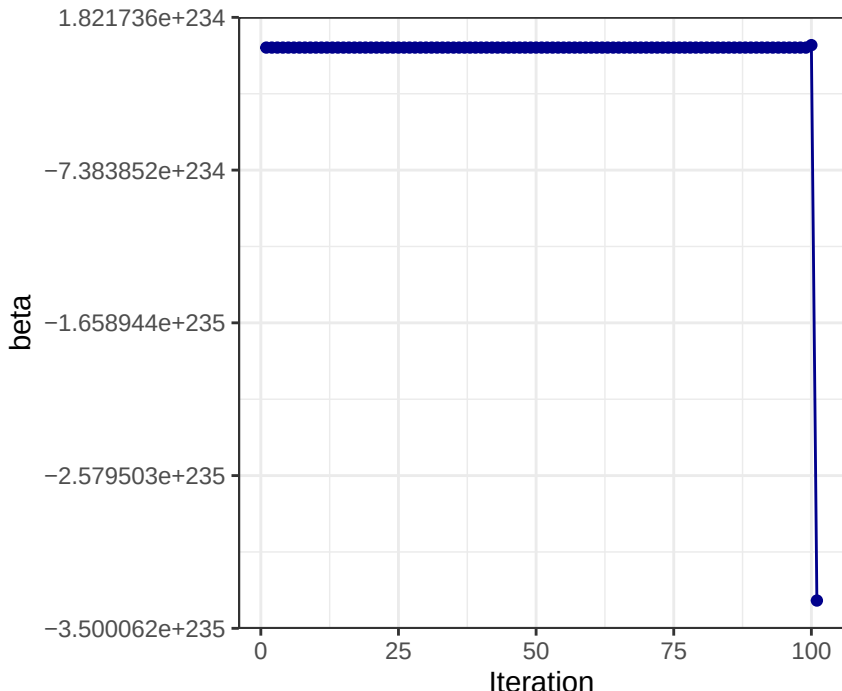
- 过大的步长

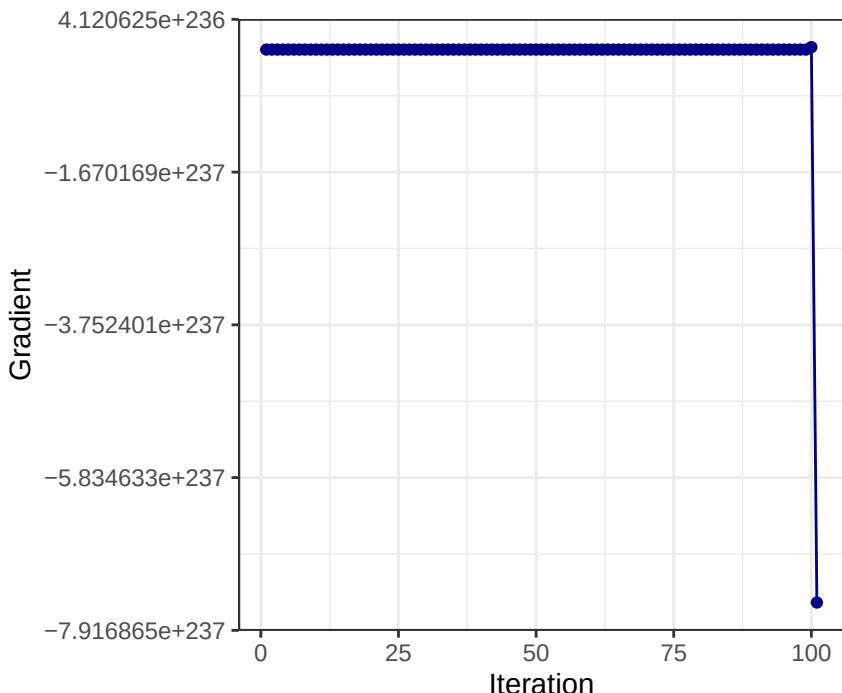
```
iter <- 0
beta <- 0

s <- 1 # stepsize
while(abs(g(beta[iter + 1]))>1e-3 & iter < 100){
  iter <- iter + 1
  beta[iter+1] <- beta[iter] - s*g(beta[iter])
}
```

```
ggplot_series(beta, xlab = "Iteration", ylab = "beta")
```

```
ggplot_series(g(beta), xlab = "Iteration", ylab = "Gradient")
```





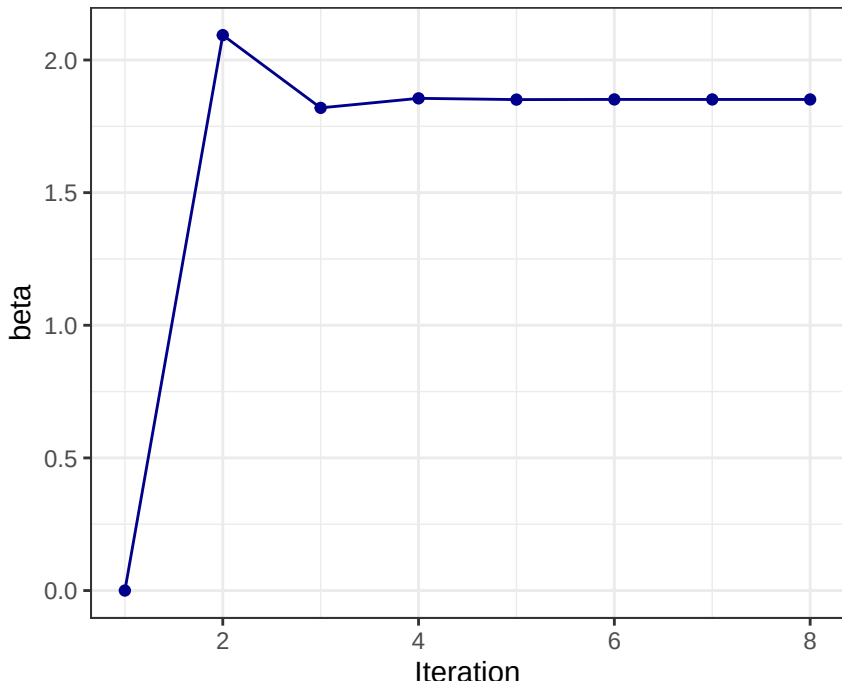
- 合适的步长

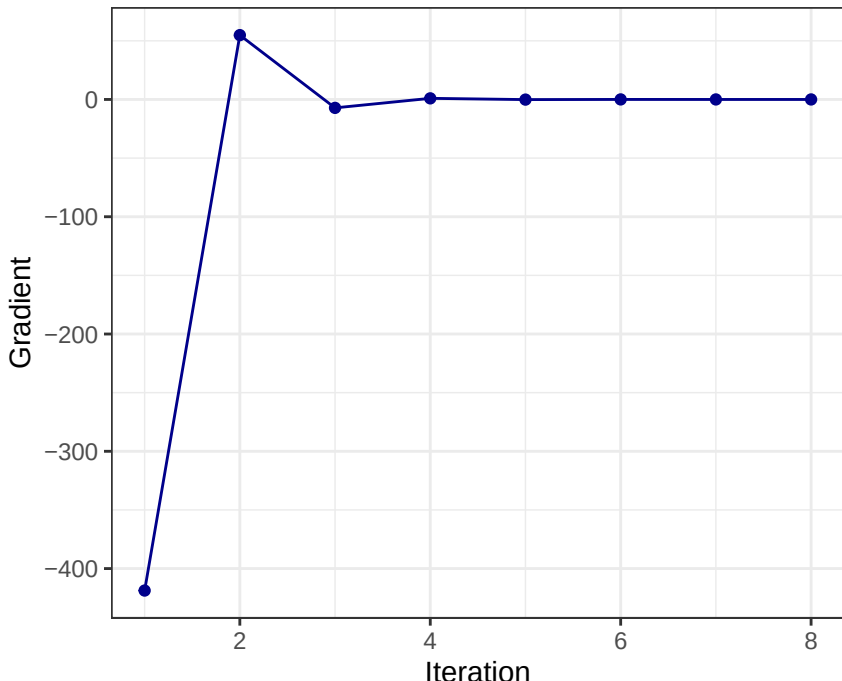
```
iter <- 0
beta <- 0

s <- 200{-1} # stepsize
while(abs(g(beta[iter + 1]))>1e-3 & iter < 100){
  iter <- iter + 1
  beta[iter+1] <- beta[iter] - s*g(beta[iter])
}
```

```
ggplot_series(beta, xlab = "Iteration", ylab = "beta")
```

```
ggplot_series(g(beta), xlab = "Iteration", ylab = "Gradient")
```





最优步长梯度下降算法 (*limited minimization rule*)

- *Question:* How to compute the optimal stepsize?

```
iter <- 0
beta <- 0

while(abs(sum(x^2)*beta - sum(x*y))>1e-3 & iter < 100){
  iter <- iter + 1
  s <- 0.5/sum(x^2)
  beta <- beta - s * 2*(sum(x^2)*beta - sum(x*y))
  cat(iter, beta, "\n")
}
```

```
## 1 1.851271
```

- *Question:* How to implement Armijo rule in this problem?

3.4 投影梯度法

- 投影下降法 (*Projected Gradient Method*) 是用来处理具有约束条件的优化算法。
- 考虑带有约束条件的优化问题:

$$x^* = \arg \min_{x \in \mathcal{C}} f(x),$$

这里 $\mathcal{C}$ 为一个约束空间。

- 回忆梯度下降法的迭代公式为

$$x_{t+1} = x_t - s_t \nabla f(x_t).$$

- 梯度下降法迭代公式的另一种等价写法为

$$x_{t+1} = \arg \min_{x \in \mathbb{R}^p} \left\{ f(x_t) + \langle \nabla f(x_t), x - x_t \rangle + \frac{1}{2s_t} \|x - x_t\|_2^2 \right\}.$$

- 投影梯度法在第 $t + 1$ 步的迭代公式为:

$$x_{t+1} = \arg \min_{x \in \mathcal{C}} \left\{ f(x_t) + \langle \nabla f(x_t), x - x_t \rangle + \frac{1}{2s_t} \|x - x_t\|_2^2 \right\}.$$

- 本质上，投影梯度法等价于先做一次梯度下降，再投影到约束空间 $\mathcal{C}$ 上 (Why?)

- 投影梯度法的迭代公式为:

$$\begin{aligned}x_{t+1} &= \arg \min_{x \in \mathcal{C}} \left\{ f(x_t) + \langle \nabla f(x_t), x - x_t \rangle + \frac{1}{2s_t} \|x - x_t\|_2^2 \right\} \\&= \arg \min_{x \in \mathcal{C}} \|x - x_t + s_t \nabla f(x_t)\|_2^2 \\&= \arg \min_{x \in \mathcal{C}} \|x - (x_t - s_t \nabla f(x_t))\|_2^2\end{aligned}$$

- 投影梯度法等价于先做一次梯度下降

$$\tilde{x}_{t+1} = x_t - s_t \nabla f(x_t),$$

再将其投影到约束空间上, 即

$$x_{t+1} = \arg \min_{x \in \mathcal{C}} \|x - \tilde{x}_{t+1}\|.$$

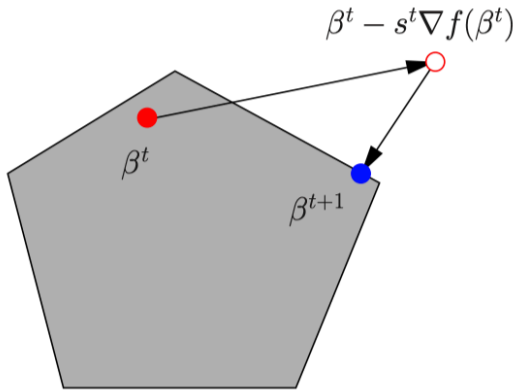
◦ 投影梯度算法:

1. 初始化 x_0 ;

2. 计算 $\tilde{x}_{t+1} = x_t - s_t \nabla f(x_t)$;

3. 更新 $x_{t+1} = \arg \min_{x \in \mathcal{C}} \|x - \tilde{x}_{t+1}\|$.

- 投影梯度法的一个图示²



思考: 如何将投影梯度法中考虑的带有约束的优化问题转化为无约束的优化问题?

²Hastie, T., Tibshirani, R., and Wainwright, M. (2015). Statistical learning with sparsity: the lasso and generalizations. CRC press.

3.5 近端梯度法

- 近端梯度法 (*Proximal gradient method*) 一般用于凸优化问题中，主要用来处理如下优化问题：

$$x^* = \arg \min_{x \in \mathbb{R}^p} f(x), \quad f(x) = g(x) + h(x),$$

其中 g 函数为凸函数且可微, h 函数为凸函数但不可微。

- 当

$$h(x) = I_{\mathcal{C}}(x) = \begin{cases} 0, & \text{if } x \in \mathcal{C}, \\ \infty, & \text{otherwise} \end{cases}$$

则该优化问题退化为

$$x^* = \arg \min_{x \in \mathcal{C}} g(x).$$

因此，近端梯度法能够处理所有投影梯度法可以处理的问题。

- 近端梯度法来源于梯度下降法中迭代公式的等价写法

$$x_{t+1} = \arg \min_{x \in \mathbb{R}^p} \left\{ f(x_t) + \langle \nabla f(x_t), x - x_t \rangle + \frac{1}{2s_t} \|x - x_t\|_2^2 \right\}.$$

对于可微部分 g 我们采取同样的近似方法，而对 h 保持不动。迭代公式为

$$\begin{aligned} & x_{t+1} \\ &= \arg \min_{x \in \mathbb{R}^p} \left\{ g(x_t) + \langle \nabla g(x_t), x - x_t \rangle + \frac{1}{2s_t} \|x - x_t\|_2^2 + h(x) \right\}. \end{aligned}$$

- 定义近端映射算子 (*proximal map operator*):

$$\text{prox}_h(y) = \arg \min_{x \in \mathbb{R}^p} \frac{1}{2} \|y - x\|_2^2 + h(x).$$

- 利用近端映射算子，近端梯度法的迭代公式可以改写为

$$x_{t+1} = \text{prox}_{s_t h}(x_t - s_t \nabla g(x_t)).$$

(How?)

- 近端梯度算法:
 - 初始化 x_0 ;
 - 计算 $\tilde{x}_{t+1} = x_t - s_t \nabla g(x_t)$;
 - 更新 $x_{t+1} = \text{prox}_{s_t h}(\tilde{x}_{t+1})$.

近端映射算子 $\text{prox}_h(x)$ 的一些特殊例子

- 当函数

$$h(x) = I_{\mathcal{C}}(x) = \begin{cases} 0, & \text{if } x \in \mathcal{C}, \\ \infty, & \text{otherwise} \end{cases}$$

则

$$\text{prox}_h(y) = \arg \min_{x \in \mathcal{C}} \|y - x\|_2^2$$

是一个投影算子，将 y 投影到空间 $\mathcal{C}$ 上；

- 当函数

$$h(x) = \lambda \|x\|_1,$$

这里 $\lambda > 0$ 为固定参数, $\|x\|_1 = \sum_{j=1}^p |x_j|$ 为 L_1 范数。则

$$\text{prox}_h(y) = \mathcal{S}_\lambda(y)$$

是一个 self-thresholding operator, 定义为

$$[\mathcal{S}_\lambda(y)]_j = \text{sign}(y_j)(|y_j| - \lambda)_+, \quad j = 1, \dots, p,$$

这里 $(y)_+ = \max\{y, 0\}$.

收敛速率

- Nesterov (2007) 证明了近端梯度法的收敛速率。假设目标函数 $f = g + h$, g 函数连续可微并且梯度满足 Lipschitz 连续, 即存在常数 $L > 0$,

$$\|\nabla g(x) - \nabla g(x')\|_2 \leq L\|x - x'\|_2, \quad \forall x, x' \in \mathbb{R}^p.$$

那么当取固定步长 $s_t = s \in (0, 1/L]$, 目标函数有如下收敛速率

$$|f(x_t) - f(x^*)| \leq \frac{C}{t+1} \|x_0 - x^*\|_2.$$

- 因此对于性质良好的目标函数, 近端梯度法可以保证目标函数具有次线性收敛速率。

- 如果进一步假设 g 函数具有强凸性 (strong convexity), 即

$$g(x + \Delta) - g(x) - \langle \nabla g(x), \Delta \rangle \geq \gamma^2 \|\Delta\|_2^2, \quad \forall x, \Delta \in \mathbb{R}^p,$$

那么近端梯度法可以保证目标函数达到线性收敛速率,

$$|f(x_t) - f(x^*)| \leq C\kappa^t \|x_0 - x^*\|_2,$$

这里 $\kappa \in (0, 1)$ 是一个常数。

例 3.4: Lasso problem

对于数据矩阵 $y \in \mathbb{R}^n$, $X \in \mathbb{R}^{n \times p}$, 考虑如下求解问题

$$\min_{\beta \in \mathbb{R}^p} \frac{1}{2n} \|y - X\beta\|_2^2 + \lambda \|\beta\|_1.$$

- 目标函数第一部分 $g(\beta) = \|y - X\beta\|_2^2 / (2n)$ 凸且可微, 第二部分 $h(\beta) = \lambda \|\beta\|_1$ 凸但不可微。考虑用近端梯度法。
- Lipschitz 常数 $L = (1/n) \|X^\top X\|$ (why?)

- 取固定步长 $s_t = 1/L$, $L = (1/n)\|X^\top X\|$.
- 迭代公式为:

$$\beta_{t+1} = \text{prox}_{s_t h}(\beta_t - s_t \nabla g(\beta_t)),$$

1. 第一步: $\tilde{\beta}_{t+1} = \beta_t - s_t \nabla g(\beta_t) = \beta_t + \frac{1}{n} s_t X^\top (Y - X\beta_t)$;
2. 第二步: $\beta_{t+1,j} = \mathcal{S}_{s_t \lambda} \{\tilde{\beta}_{t+1,j}\}$, $j = 1, \dots, p$.

- 低维数据: $p = 8$

```
# Generate data
set.seed(2)
n <- 500
p <- 8
x <- matrix(rnorm(n*p), n, p)
beta <- c(3, 1.5, 0, 0, 2, 0, 0, 0)
y <- x%*%beta + rnorm(n)

soft <- function(x, lam){
  ifelse(abs(x)>lam, sign(x)*(abs(x)-lam), 0)
}
```

```
# Initialization
```

```
b.old <- b <- rep(0, 8)
```

```
cvg <- c(1)
```

```
iter <- 0
```

```
lambda <- 0.1
```

```
tol <- sqrt(.Machine$double.eps)
```

```
step <- n/(svd(t(x)%*%x)$d[1])
```

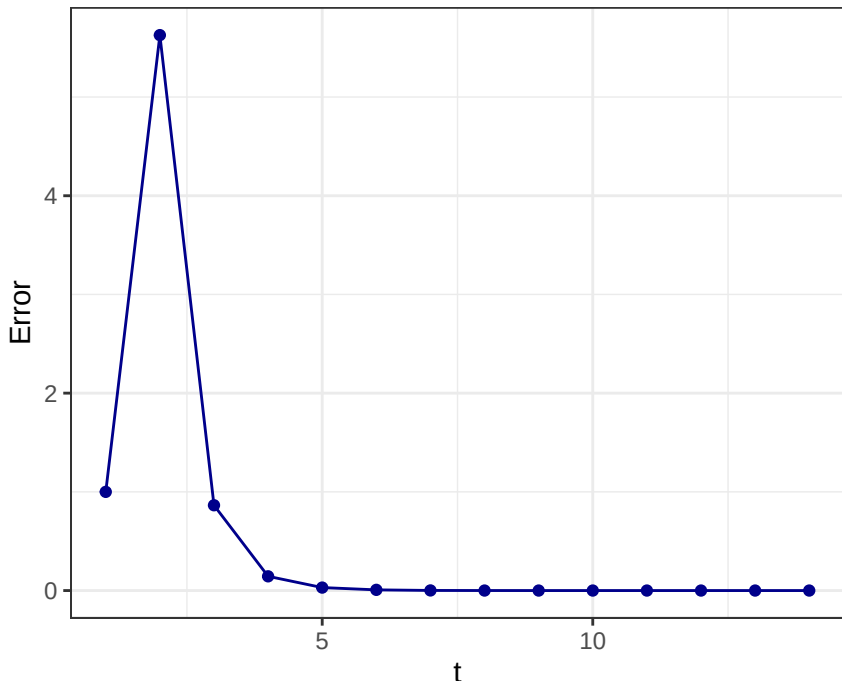
```
step
```

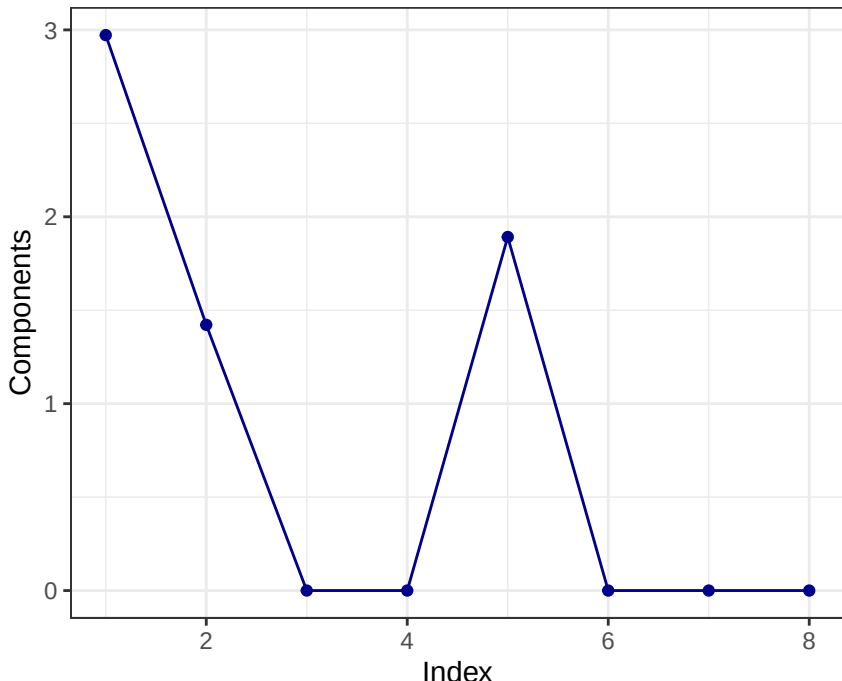
```
## [1] 0.8500063
```

```
options(width = 50)
while(cvg[iter+1] > tol && iter < 100){
  iter <- iter + 1
  b <- b.old + (1/n)*step*t(x)%*(y - x%*b.old)
  b <- sapply(b, soft, lambda*step)
  diff <- sum(abs(b-b.old))
  cvg <- c(cvg, diff)
  b.old <- b
}
```

```
ggplot_series(cvg, xlab = "t", ylab = "Error")
```

```
ggplot_series(b.old, xlab = "Index", ylab = "Components")
```





- 高维数据: $p = 300$

```
set.seed(2)
n <- 100
p <- 200
x <- matrix(rnorm(n*p), n, p)
beta <- rep(0, p)
beta[1:10] <- 2
y <- x%*%beta + rnorm(n)
```

Initialization

b.old <- b <- rep(0, p)

cvg <- c(1)

iter <- 0

lambda <- 0.8

tol <- sqrt(.Machine\$double.eps)

step <- n/(svd(t(x)%*%x)\$d[1])

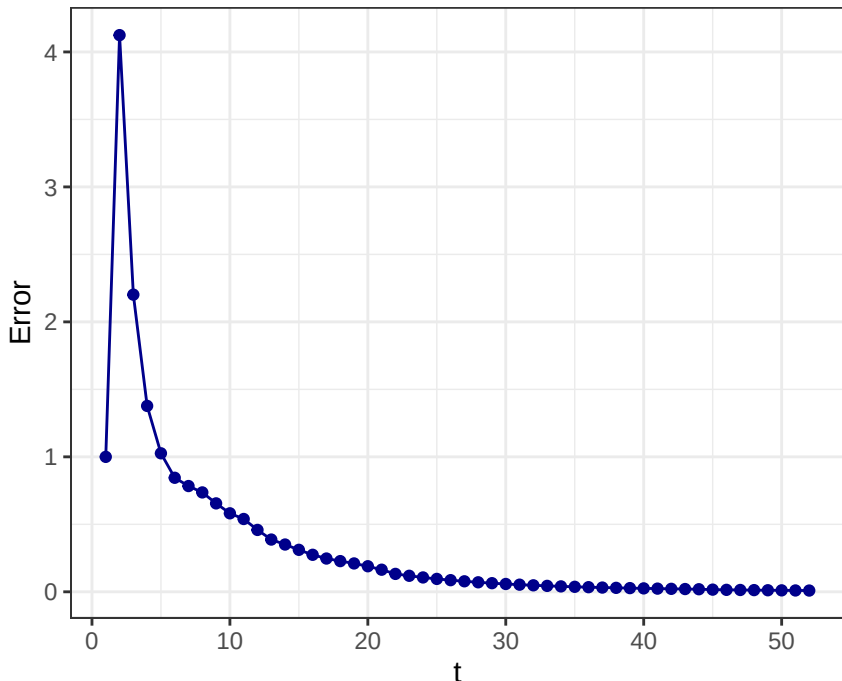
```
while(cvg[iter+1] > 1e-2 && iter < 100){  
  iter <- iter + 1  
  b <- b.old + (1/n)*step*t(x)%*%(y - x)%*%b.old)  
  b <- sapply(b, soft, lambda*step)  
  diff <- sum(abs(b-b.old))  
  cvg <- c(cvg, diff)  
  b.old <- b  
}
```

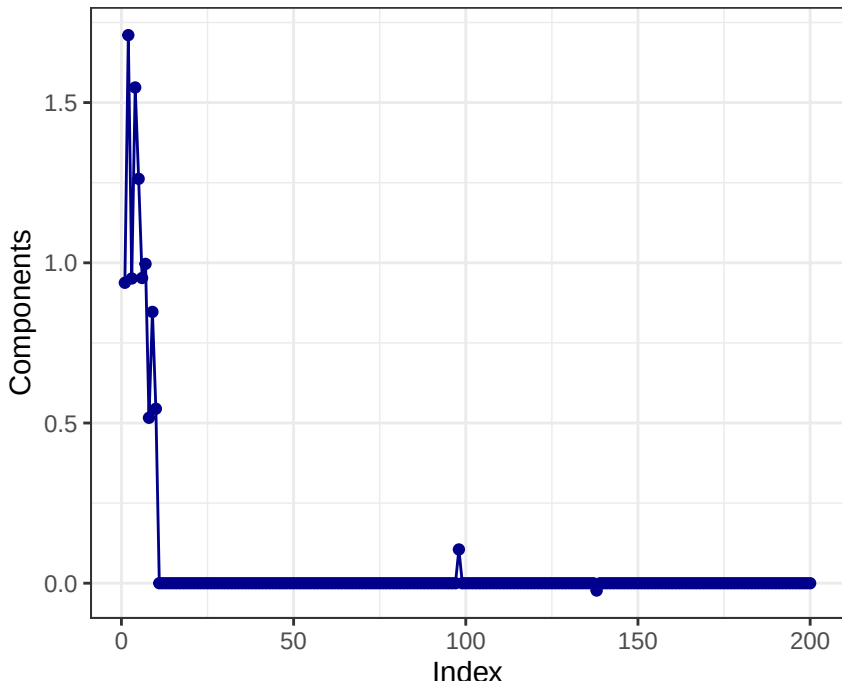
```
ggplot_series(cvg, xlab = "t", ylab = "Error")
```

```
ggplot_series(b.old, xlab = "Index", ylab = "Components")
```

Note: 需要通过 data-driven(例如交叉验证) 的方法来调整参数 λ .

思考: 参数 λ 是如何影响 β 估计量的稀疏性的?





3.6 坐标下降法

- 坐标下降法 (*Coordinate descent*): 在每一轮中对于多元变量的单个变量逐个更新。
- 具体来说, 对于优化问题

$$x^* = \arg \min_{x \in \mathbb{R}^p} f(x),$$

我们在第 t 次迭代中依次更新第 j 个变量, $j = 1, \dots, p$:

$$x_{t+1,j} = \arg \min_{x \in \mathbb{R}} f(x_{t+1,1}, \dots, x_{t+1,j-1}, x_{t,j+1}, \dots, x_{t,p}).$$

一轮下来, 我们得到一组新的变量

$$x_{t+1} = (x_{t+1,1}, \dots, x_{t+1,p}).$$

收敛性: 假设目标函数形式具有如下可分结构,

$$f(x_1, \dots, x_p) = g(x_1, \dots, x_p) + \sum_{j=1}^p h_j(x_j),$$

其中 g 是可微凸函数, h_j 为凸函数但是不要求可微。Tseng (1988)³, Tseng (2001)⁴证明了对于这种可分结构的目标函数, 坐标下降法一定收敛。

- Lasso 问题中目标函数满足可分结构:

$$f(\beta) = \frac{1}{2n} \|y - X\beta\|_2^2 + \lambda \sum_{j=1}^p |\beta_j|.$$

³Tseng, P. (1988), Coordinate ascent for maximizing nondifferentiable concave functions, Technical Report LIDS-P ; 1840, Massachusetts Institute of Technology. Laboratory for Information and Decision Systems.

⁴Tseng, P. (2001), Convergence of block coordinate descent method for nondifferentiable maximization, Journal of Optimization Theory and Applications 109(3), 474-494.

例 3.4 (续) : Lasso problem

- 固定 β_j 外的其他变量, 则目标函数为一个一维的 Lasso 问题

$$f(\beta_j) = \frac{1}{2n} \|(y - X_{-j}\beta_{-j}) - X_j\beta_j\|_2^2 + \lambda|\beta_j| + C,$$

这里 X_{-j} 为从数据阵 X 除去第 j 列得到的矩阵, β_{-j} 同理, C 为与 β_j 无关的常数。

- $f(\beta_j)$ 的最小值为 (how?)

$$\hat{\beta}_j = \mathcal{S}_{\frac{n\lambda}{\|X_j\|_2^2}} \left(\frac{(y - X_{-j}\beta_{-j})^\top X_j}{\|X_j\|_2^2} \right)$$

```
# Generate data
set.seed(1)
n <- 100
p <- 8
x <- matrix(rnorm(n*p), n, p)
beta <- c(3, 1.5, 0, 0, 2, 0, 0, 0)
y <- x%*%beta + rnorm(n)

# The norm square of each column of data matrix X
xn <- apply(x^2, 2, sum)

# The soft-thresholding operator
soft <- function(x, lam){
  ifelse(abs(x)>lam, sign(x)*(abs(x)-lam), 0)
}

# Initialization
b.old <- b <- rep(0, 8)
cvrg <- c(1)
```

```
iter <- 0
lambda <- 0.1
tol <- sqrt(.Machine$double.eps)

while(cvg[iter+1] > tol && iter < 100){
  iter <- iter + 1
  for(j in 1:p){
    z <- y - x[,-j]%*%b[-j]
    b[j] <- soft(sum(x[,j]*z)/xn[j], n*lambda/xn[j])
  }
  diff <- sum(abs(b-b.old))
  cvg <- c(cvg, diff)
  b.old <- b
}
```

```
options(width = 50)
```

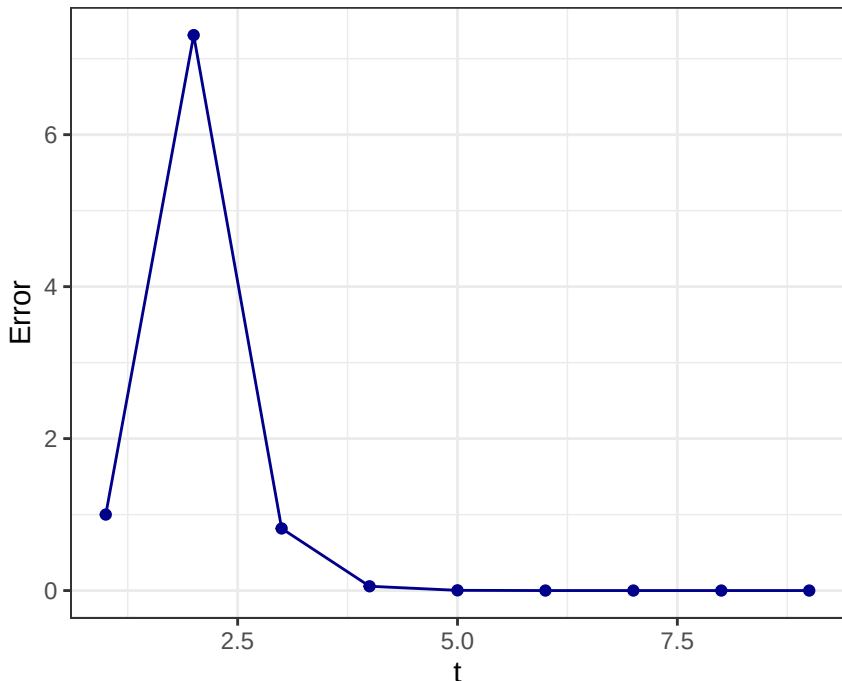
```
b.old # beta
```

```
## [1] 2.97028977 1.39565318 0.00000000 0.13616216
```

```
## [5] 2.01998563 0.07434669 0.00000000 0.00000000
```

```
ggplot_series(cvg, xlab = "t", ylab = "Error")
```

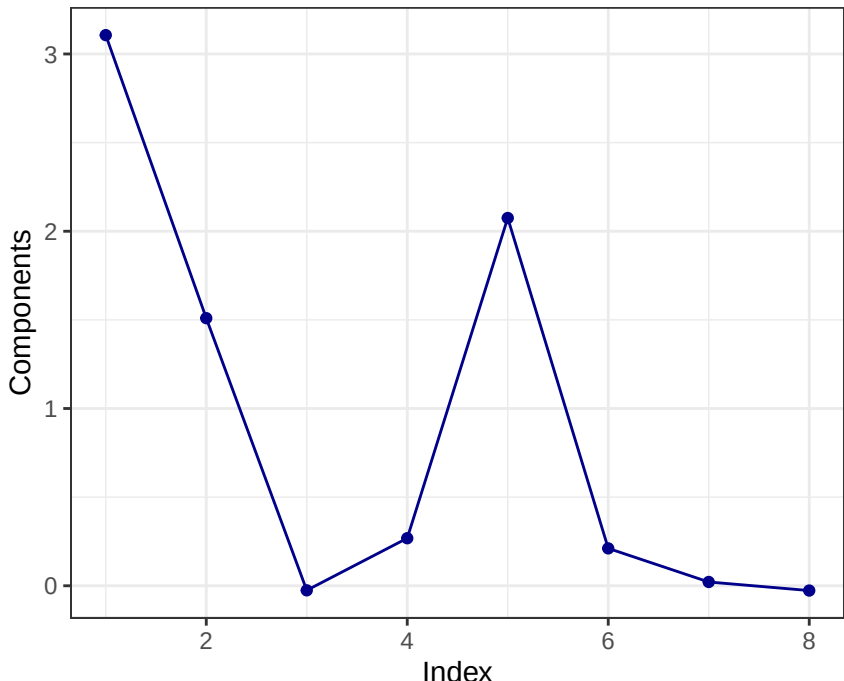
Note: 需要选择一个合适的 λ



- 取较小 λ

```
lambda <- 0.001  
b.old <- b <- rep(0, 8)  
cvg <- c(1)  
iter <- 0
```

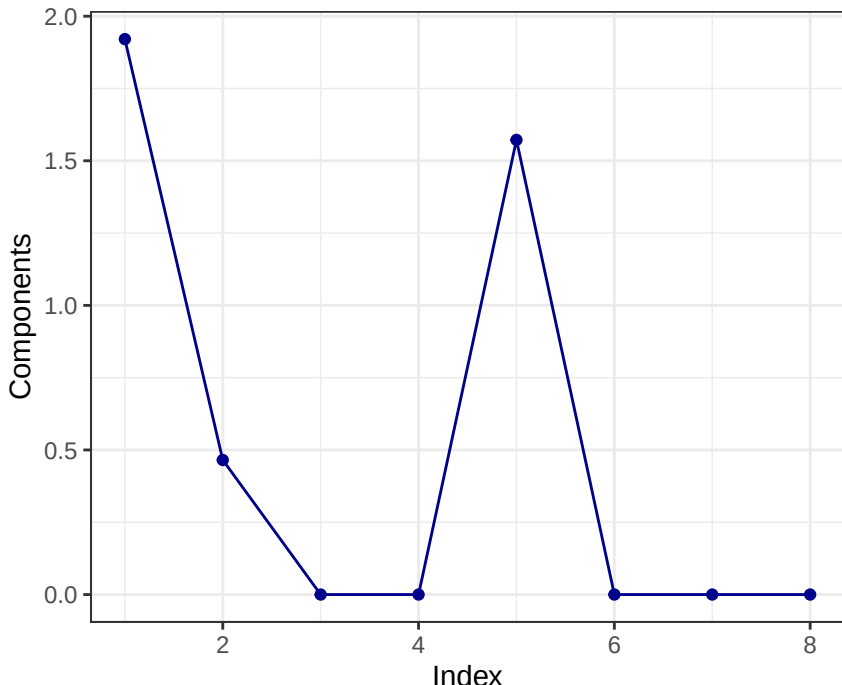
```
options(width = 50)
while(cvg[iter+1] > tol && iter < 100){
  iter <- iter + 1
  for(j in 1:p){
    z <- y - x[,-j]%*%b[-j]
    b[j] <- soft(sum(x[,j]*z)/xn[j], n*lambda/xn[j])
  }
  diff <- sum(abs(b - b.old))
  cvg <- c(cvg, diff)
  b.old <- b
}
ggplot_series(b.old, xlab = "Index", ylab = "Components")
```



- 取较大 λ

```
lambda <- 1  
b.old <- b <- rep(0, 8)  
cvg <- c(1)  
iter <- 0
```

```
while(cvg[iter+1] > tol && iter < 100){  
  iter <- iter + 1  
  for(j in 1:p){  
    z <- y - x[,-j]%*%b[-j]  
    b[j] <- soft(sum(x[,j]*z)/xn[j], n*lambda/xn[j])  
  }  
  diff <- sum(abs(b - b.old))  
  cvg <- c(cvg, diff)  
  b.old <- b  
}  
ggplot_series(b.old, xlab = "Index", ylab = "Components")
```



- 取合适 λ

```
lambda <- 0.22  
b.old <- b <- rep(0, 8)  
cvg <- c(1)  
iter <- 0
```

```
while(cvg[iter+1] > tol && iter < 100){  
  iter <- iter + 1  
  for(j in 1:p){  
    z <- y - x[,-j]%*%b[-j]  
    b[j] <- soft(sum(x[,j]*z)/xn[j], n*lambda/xn[j])  
  }  
  diff <- sum(abs(b - b.old))  
  cvg <- c(cvg, diff)  
  b.old <- b  
}  
ggplot_series(b.old, xlab = "Index", ylab = "Components")
```

